

GUIA RESUMEN — Git Flow y Conventional Commits

Resumen integral para dominar el flujo de trabajo en equipo en ~20 minutos, sin volver a leer el modulo completo.

0

Flujo de aprendizaje general

5 PARTES · ~20 MIN

1 Commits 5 min	2 Hooks 3 min	3 Asistentes 3 min	4 FVM 2 min	5 Ramas 5 min
--	--	---	--	--

Regla de oro: "Los commits comunican QUÉ cambió. Los tags comunican CUÁNDO se liberó. El CHANGELOG resume todo para el equipo y usuarios."

Parte	Que aprendes	Archivo clave
1. Conventional Commits	Formato de commits, tipos, breaking changes	01-conventional-commits.md
2. Husky + lint-staged	Hooks de pre-commit, commit-msg, pre-push	02-husky-lint-staged.md
3. Commitizen + commitlint	CLI interactivo, validador, CHANGELOG auto	03-commitizen-commitlint.md
4. FVM	Gestion de versiones de Flutter por proyecto	04-fvm-version-management.md
5. Ramas y Releases	Git Flow, Conventional Branch, SemVer	05-flujo-ramas-estrategias.md

Conventional Commits = convencion ligera para mensajes de commit que permite generar CHANGELOGs y versionado semantico automatico.

Formato del mensaje

```
<tipo>[<ambito opcional>][<! opcional>]: <descripcion>

[cuerpo opcional]

[nota(s) al pie opcional(es)]
```

Tipos principales

Tipo	Que hace	Version SemVer
feat	Nueva funcionalidad	MINOR (1.0.0 → 1.1.0)
fix	Correccion de bug	PATCH (1.0.0 → 1.0.1)
BREAKING CHANGE	Cambio incompatible	MAJOR (1.0.0 → 2.0.0)

Tipos adicionales

Tipo	Cuando usar	Ejemplo
build	Build system	build: upgrade gradle a 8.0
chore	Mantenimiento	chore: limpiar dependencias
ci	CI/CD	ci: agregar step de codegen
docs	Documentacion	docs: actualizar README
perf	Rendimiento	perf: cachear consultas
refactor	Refactorizar	refactor: extraer widget
style	Formato	style: dart format
test	Tests	test: cubrir caso borde

Reglas de oro

- ✓ Imperativo presente ("agregar", no "agregue")
- ✓ Sin punto final
- ✓ Maximo 72 caracteres
- ✓ Primera letra minuscula
- ✓ Ambito opcional: feat(auth): ...

Breaking Changes

```
# Opcion 1: ! despues del tipo
refactor!(auth): cambiar respuesta de login a JWT

# Opcion 2: BREAKING CHANGE en el pie
feat: migrar de SQLite a Isar

BREAKING CHANGE: Se elimino la base de datos local.
```

Ejemplo real: feat(raffles): agregar filtro por fecha → version 1.0.0 → 1.1.0. Si incluye ! o BREAKING CHANGE → 1.1.0 → 2.0.0

2 Husky + lint-staged (3 min)

HOOKS DE GIT

Husky ejecuta scripts automaticos en hooks de Git. **lint-staged** ejecuta comandos SOLO en archivos staged.

Los 3 hooks principales

Hook	Cuando se ejecuta	Que hace
pre-commit	Antes del commit	Formateo + analisis (lint-staged)
commit-msg	Despues del mensaje	Valida formato (commitlint)
pre-push	Antes del push	Ejecuta tests completos

Configuracion lint-staged para Flutter

```
"lint-staged": {
  "**.dart": [
    "dart format --set-exit-if-changed",
    "flutter analyze --fatal-infos"
  ],
  "**.{yaml,json}": ["prettier --write"]
}
```

Emergencias: `git commit --no-verify -m "fix: hotfix critico"` **salta los hooks temporalmente.**

3 Commitizen + commitlint (3 min)

ASISTENTES DE COMMIT

Commitizen = CLI interactivo que genera commits validos. **commitlint** = linter que valida mensajes.

Flujo con Commitizen

```
$ npx cz

? Selecciona el tipo de cambio: feat
? Alcance: raffles
? Descripcion: agregar filtro por fecha
? Breaking change: No

→ feat(raffles): agregar filtro por fecha
```

Reglas de commitlint

Regla	Valor	Descripcion
type-enum	[2, 'always', [...]]	Tipos permitidos
type-empty	[2, 'never']	Tipo requerido
subject-empty	[2, 'never']	Descripcion requerida
header-max-length	[2, 'always', 72]	Maximo 72 caracteres
subject-full-stop	[2, 'never', '.']	Sin punto final

Standard Version (CHANGELOG automatico)

```
# Ejecutar release
npm run release

# Esto genera:
# - CHANGELOG.md actualizado
# - package.json con version bump
# - Commit con los cambios
# - Tag v1.2.0
```

Tabla resumen: **Commitizen (escribe) + commitlint (valida) + standard-version (libera) = flujo completo.**

FVM permite que cada proyecto tenga su propia version exacta de Flutter.

Comandos esenciales

Comando	Que hace
<code>fvm install 3.24.0</code>	Instala una version
<code>fvm use 3.24.0</code>	Configura el proyecto
<code>fvm flutter pub get</code>	Usa la version del proyecto
<code>fvm list</code>	Lista versiones instaladas
<code>fvm current</code>	Version actual del proyecto

Archivos clave

```
.fvm/flutter_sdk_version # Version configurada (SE COMITA)
.fvm/flutter_sdk        # SDK instalado (SE IGNORA en .gitignore)
```

VS Code: Configurar `"dart.flutterSdkPath": ".fvm/flutter_sdk"` en `.vscode/settings.json` para que use la version correcta automaticamente.

Conventional Branch = nomenclatura estructurada para ramas Git.

Prefijos de ramas

Prefijo	Proposito	Ejemplo
main	Rama principal (produccion)	main
feature/	Nuevas funcionalidades	feature/agregar-login
bugfix/	Corregir errores	bugfix/corregir-crash
hotfix/	Correcciones urgentes	hotfix/parche-seguridad
release/	Preparar liberacion	release/v1.2.0
chore/	Tareas sin codigo	chore/actualizar-deps

Git Flow simplificado

```
main — M1 — M2 — M3 (produccion)
      |
develop — D1 — D2 — D3 — D4 — D5 — D6 (integracion)
      |
feature/ |
         | feat1
         |
         | feat2
         |
         | feat3
```

Versionado Semantico (SemVer 2.0.0)

Version	Cuando incrementar	Ejemplo
Major (X)	Cambios incompatibles	1.0.0 → 2.0.0
Minor (Y)	Nueva funcionalidad	1.0.0 → 1.1.0
Patch (Z)	Correccion de bug	1.1.0 → 1.1.1

Flujo completo

1. Crear rama feature desde develop
2. Commits con feat/fix (Conventional Commits)
3. PR a develop + code review
4. Merge a develop
5. Crear release/X.Y.Z desde develop
6. standard-version genera CHANGELOG + bump
7. Merge a main + tag vX.Y.Z
8. Deploy a Play Store / App Store

Hotfix: Si hay bug critico en produccion → crear hotfix/ desde main → fix → merge a main Y develop → tag vX.Y.Z+1

A Anexo · Indice del modulo (para profundizar)

Necesitas	Archivo
Especificacion completa de Conventional Commits	01-conventional-commits.md
Hooks de Git con Husky y lint-staged	02-husky-lint-staged.md
Commitizen interactivo y commitlint validador	03-commitizen-commitlint.md
FVM para gestion de versiones de Flutter	04-fvm-version-management.md
Git Flow, Conventional Branch y SemVer	05-flujo-ramas-estrategias.md

Conclusion

El flujo de trabajo profesional en equipo se basa en **convenciones claras**:

- ✓Conventional Commits para mensajes consistentes
- ✓Husky + lint-staged para calidad automatica
- ✓Commitizen + commitlint para commits validos
- ✓FVM para versiones de Flutter consistentes
- ✓Git Flow + Conventional Branch para ramas organizadas
- ✓SemVer para versionado predecible
- ✓standard-version para CHANGELOG automatico

Tiempo de lectura del resumen: ~20 min · El camino hacia un flujo de trabajo profesional empieza con convenciones claras.